

# 大数据原理与技术——期末大作业报告

李天扬 艾清 刘浩 [liuh393@mail2.sysu.edu.cn](mailto:liuh393@mail2.sysu.edu.cn)

## 大数据原理与技术——期末大作业报告

### 一，系统概述

#### 1.1 文档概述

#### 1.2 系统目标

#### 1.3 系统特色

#### 1.4 系统架构

### 二，算法设计思路

#### 2.1 整体流程与框架

#### 2.2 数据预处理

#### 2.3 字（词）向量与编码

#### 2.4 文本向量与特征工程

#### 2.5 分类器

##### 2.5.1 模型结构

##### 2.5.2 训练集样本平衡与划分

##### 2.5.3 训练与推理配置

##### 2.5.4 分类器配置表

### 三，系统优化设计

#### 3.1 性能优化

#### 3.2 轻量化版本设计

#### 3.3 前端配套设计

### 四，实验结果与分析

#### 4.1 测试方案与指标说明

#### 4.2 效果展示

#### 4.3 结果分析

#### 4.4 消融实验与分析

##### 4.4.1 无结构特征 vs 有结构特征（特征工程）

##### 4.4.2 样本平衡

##### 4.4.3 低维PCA vs 高维PCA

##### 4.4.4 普通逻辑回归分类器和神经网络分类器

##### 4.4.5 优化性能用时（端侧版本和高性能CPU+GPU版本）

##### 4.4.6 端侧训练推理模型设计

### 五，创新点与总结

#### 可能的进一步优化点

## 一，系统概述

### 1.1 文档概述

- 本章进行整个项目的概述概述
- 第二章，算法设计与思路，详细讲解垃圾文本分类算法的流程与各个模块。
- 第三章，系统优化设计，对系统的性能、不同资源场景的适应性、用户可用性进行改善。
- 第四章，实验结果与分析，对算法和模型进行检验，并根据结果进行分析。并进行消融实验，分析各个优化的效果。

- 第五章，创新点与总结，回顾系统的创新点并进行总结。

## 1.2 系统目标

垃圾文本分类（Spam Text Classification）是自然语言处理（NLP）和文本挖掘领域的一个重要任务，旨在自动识别和过滤不相关、恶意或有害的文本信息。它在多个应用场景中具有广泛的实际价值，如电子邮件过滤、社交媒体内容审核、广告检测、欺诈信息拦截等。

本项目旨在实现简单高效的中文垃圾文本分类系统，尽可能提高不同种类垃圾文本的辨别效果。

## 1.3 系统特色

- **结合垃圾文本数据集特点的特征工程。** 文本向量化时，除了计算语义特征向量，还引入了结构特征向量，提高了文本特征之间的差异，对文本特征的描述更加完整。
- **高性能的计算框架。** 针对不同模块和流程进行了一定的性能优化，如实现缓存机制和多核并行文本向量化。
- **极高的准确率和召回率，较好的模型效果。** 在多个数据集场景下均达到99%+的正确率和召回率。
- **鲁棒性好。** 针对数据集未编码字符OOV进行了针对化处理。针对数据集正负样本失衡，采用了 `RandomOverSampler` 的设计。
- **可视化强。** 增加了可视化网页进行测试，可以更加直观的评估模型性能。
- **场景适应力强。** 提供了CPU/GPU两个版本，针对不同的计算资源场景，拥有更广泛的适用性。

## 1.4 系统架构

分为前端（可视化层）+ 后端（完整的垃圾文本分类算法）

在代码组织上，不同的模块被划分为不同的python文件中，main文件调用不同的模块，完成完整的垃圾文本分类算法流程。

文件架构如下：

```
1 | project
2 | └─ **核心代码文件**
3 |   └─ app.py                # web应用入口（Flask/Django等）
4 |   └─ main.py                # 主程序入口（完整版）
5 |   └─ main_light.py          # 轻量化主程序入口
6 |   └─ character_coder.py      # 汉字编码处理
7 |   └─ data_processing.py      # 数据预处理模块
8 |   └─ similarity_matrix.py    # 相似度矩阵计算
9 |   └─ text_to_embedding.py    # 文本向量化模块
10 |
11 | └─ **数据文件**
12 |   └─ Data/                  # 原始数据目录
13 |     └─ dataset.txt          # 原始数据集
14 |     └─ dataset_new.txt      # 新扩展数据集
15 |     └─ merged_dataset.txt   # 合并后的数据集
16 |     └─ chinese_characters_code.txt # 汉字编码表（完整版）
```

```

17 | | | └─ chinese_characters_code_light.txt # 轻量化编码表
18 | | | └─ hanzijiegou_2w.txt # 汉字结构数据
19 | | | └─ hit_stopwords.txt # 停用词表
20 | | | └─ similarity_matrix.pkl # 完整相似度矩阵
21 | | | └─ similarity_matrix_light.pkl # 轻量化相似度矩阵
22
23 |─ **模型文件**
24 | |─ mlp_model.pth # PyTorch模型权重
25 | |─ model/ # 模型相关文件
26 | | |─ model.pkl # 序列化模型（如sklearn）
27 | | |─ char_embeddings.npy # 字符嵌入向量
28 | | |─ char2idx.pkl # 字符到索引映射
29 | | |─ label_encoder.pkl # 标签编码器
30 | | |─ pca.pkl # PCA降维模型
31 | | |─ scaler.pkl # 特征缩放器
32
33 |─ **子模块**
34 | |─ four_corner_method/ # 四角编码子模块
35 | | |─ __init__.py # 模块初始化
36 | | |─ data/ # 四角编码数据
37 | | |─ __pycache__/ # 编译缓存
38
39 |─ **web相关**
40 | |─ templates/ # HTML模板
41 | | |─ index.html # 前端页面
42
43 |─ **文档**
44 | |─ readme.md # 项目说明文档

```

## 二，算法设计思路

### 2.1 整体流程与框架

核心算法分为四个模块，形成端到端的分类流水线：

#### 1. 数据预处理

- 加载原始文本，进行清洗与标准化
- 统计汉字频率分布，构建基础字典

#### 2. 字向量与编码

- 基于基础字典进行汉字编码
- 基于汉字编码结果构建相似性矩阵
- 通过PCA降维生成300维字向量

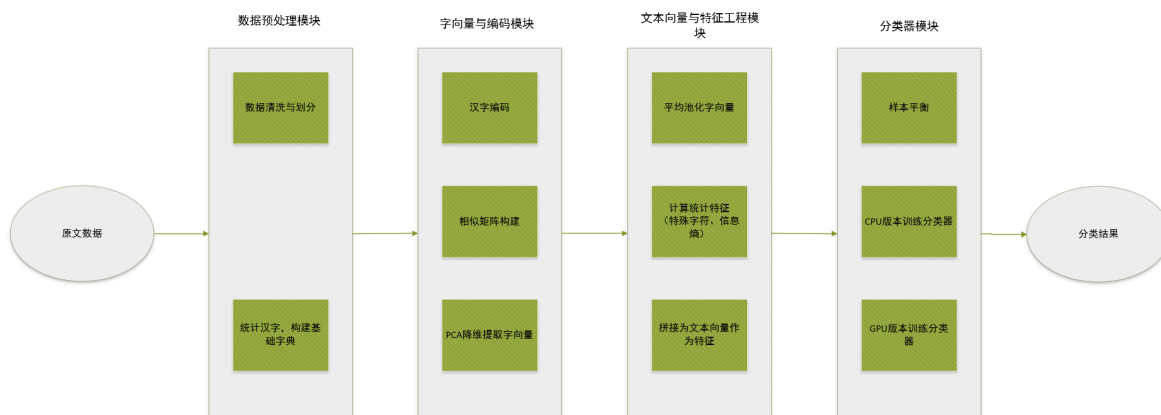
#### 3. 文本向量与特征工程

- 语义部分：平均池化字向量
- 统计特征

- 特殊符号频率（如！、¥）
- 信息熵与自相似性计算

#### 4. 分类器

- 双层全连接神经网络
- 支持CPU/GPU双版本
  - CPU版：适合端侧训练与推理
  - GPU版：CUDA加速训练与推理



## 2.2 数据预处理

从原始文本中提取标签与内容，规范化样本结构，并统计核心语言单位（汉字）以支持后续编码与建模操作。

- 步骤1：加载与划分原始数据集
  - 使用 `divide_dataset(filename, lines=...)` 函数加载原始文本数据；
  - 每行样本按 `标签\t内容` 格式组织；
  - 对不符合格式的样本进行过滤（如缺失字段、空行）；
  - 分离标签列表与文本内容，输出 `tags` 与 `texts` 两个列表；
  - 同时统计标签集合，便于后续标签编码与分布观察。
- 步骤2：提取文本中的中文汉字
  - 遍历所有文本内容，逐字符识别是否为汉字（范围 `\u4e00~\u9fff`）；
  - 构建一个字频统计字典 `chinese_characters_count`；
  - 所有出现过的汉字组成汉字表 `chinese_characters`，用于编码与建模。

```
1 def divide_dataset(filename, lines=10000):
2     with open(filename, "r", encoding="utf-8") as f:
3         text_data = f.readlines()
4         subset = text_data[:lines]
5         dataset = []
6         for i, line in enumerate(subset):
7             parts = line.strip().split("\t")
8             if len(parts) < 2 or not parts[1].strip():
9                 print(f"[跳过] 第 {i} 行数据格式错误: {line.strip()}")
```

```

10         continue
11         dataset.append(parts)
12     tag = [data[0] for data in dataset]
13     text = [data[1] for data in dataset]
14     print("Dataset size: ", len(dataset))
15     print("Tags: ", set(tag))
16     return tag, text

```

## 2.3 字（词）向量与编码

将每个汉字编码为一个结构性特征向量，通过比较这些编码生成**字符间相似度矩阵**，再用**PCA降维**将每个字符转化为**低维词向量**，用于文本分类等下游任务。

- 步骤1：为统计的汉字生成编码特征
  - 拼音编码
    - 使用 `pypinyin` 提取拼音（含声调）；划分为声母（initial）、韵母（final）、补码（coda）和声调（tone）；将这些部分映射为字符编码。
  - 字形编码
    - 结构信息。从 `hanzijiegou_2w.txt` 文件读取，如“左右结构”“上下结构”等；每种结构用单字符编码表示。
    - 四角编码。使用 `FourCornerMethod().query(hanzi)` 得到类似“3141”这样的结构代码，表示字形外观布局；
    - 笔画数。从 `chinese_unicode_table.txt` 获取笔画数并映射为单字符；如 1~51 的笔画数映射为 `1-9A-Z...p`（共52个字符）。
  - 最终将拼音、结构信息、四角编码、笔画数拼接成一个完整的字符串作为每个汉字的“唯一结构性编码”
- 步骤2：计算字符相似性矩阵
  - 构建一个空的相似度矩阵，大小为  $n * n$ ， $n$  为汉字数量
  - 使用字符串比较方法（如编辑距离、匹配分数）对每对汉字编码进行比较；
  - 相似度越高，说明两个汉字在语言结构上越接近；
  - 保证矩阵对称性：`sim_mat[i][j] = sim_mat[j][i]`。
  - 将矩阵持久化为 `.pkl` 文件方便重复使用。
- 步骤3：PCA降维生成词向量
  - 将相似度矩阵的每一行视为一个高维特征向量，通过PCA提取主要成分，形成紧凑的**词向量嵌入**。
  - 构建字向量映射表：建立 `char2idx` 字典，字符 → 行索引；建立 `char_embeddings`，行索引 → 词向量。

这里采用的“**基于汉字结构编码 + 相似度矩阵 + PCA词向量**”方法，虽然看起来比直接用 Word2Vec 更加“手工”，但是性能较好，且对于垃圾文本分类这个任务具有相当的实际性和适应性，针对垃圾文本数据集的特点，这样的处理方式有以下优点：

| 优势      | 说明                 |
|---------|--------------------|
| 不依赖语料   | 冷启动场景也能使用          |
| 结构鲁棒性   | 识别变形字、形近字、音近字扰动    |
| 可解释性强   | 每一步编码、相似度、降维都有明确定义 |
| 适合短文本   | 字级建模粒度适合碎片化文本      |
| 工程灵活    | 编码逻辑可扩展、微调、手工增强    |
| 语义不易被规避 | 较抗对抗式干扰和绕过攻击       |

（当然，通过分词先转换成词组再进行编码也是一种方法，但是考虑到只有字符的编码数据，此处只采用字符编码的方式）

```

1  # 2. 加载或统计汉字及其编码 [音] + [结构 + 四角编码 + 笔画数]
2  try:
3      chinese_chars, _, char_codes =
4      load_chinese_characters("Data/chinese_characters_code.txt")
5  except FileNotFoundError:
6      chinese_chars, _, char_codes = count_chinese_characters(texts,
7      "Data/chinese_characters_code.txt") #生成汉字编码
8
9  # 3. 加载或计算汉字相似度矩阵 计算字符之间的相似性矩阵来表示网络关系。
10 try:
11     sim_mat = load_sim_mat("Data/similarity_matrix.pkl")
12 except FileNotFoundError:
13     sim_mat = compute_sim_mat(chinese_chars, char_codes)
14
15 # 4. 对相似度矩阵进行PCA降维（得到每个汉字的向量表示）
16 pca = PCA(n_components=256) #128,192,256均可尝试
17 char_embeddings = pca.fit_transform(sim_mat)

```

## 2.4 文本向量与特征工程

将每条文本转化为结构化特征向量，融合汉字语义信息与统计特征，为分类模型提供高质量输入表示。

本阶段通过字符级词向量与统计属性，构造文本级特征，兼顾**语义表达能力**与**结构扰动鲁棒性**，特别适用于短文本垃圾识别任务。

- 步骤1：语义向量提取（平均池化）
  - 利用 `text_to_embedding()` 函数对每条文本进行编码；
  - 提取文本中所有在字向量表 `char_embeddings` 中可识别的汉字；
  - 查找对应 PCA 向量，使用 **平均池化（mean pooling）** 聚合为固定维度的语义表示向量。

- **目的**：压缩变长文本为统一长度语义表示，支持后续模型训练。
- 步骤2：特殊字符统计（结构扰动特征）
  - 利用正则表达式提取非中英文和数字的字符（如：`! ¥ @#~`）；
  - 计算：
    - **特殊符号占比**（符号数 / 总字符数）；
    - **特殊符号数量**（绝对数值）。
  - **目的**：识别垃圾文本中常见的**异常符号使用模式**，如广告推广、口号堆砌。
- 步骤3：信息熵与自相似性分析（内容分布特征）
  - **信息熵** `text_entropy(text)`：
    - 基于字符频率分布计算；
    - 反映内容复杂度和重复程度；
    - 文本越重复，熵值越低。
  - **自相似性** `self_similarity(text)`：
    - 将文本切分为多个重叠子块；
    - 计算子块间的 TF-IDF 相似度矩阵均值；
    - 高自相似性通常意味着**重复刷屏或模板化内容**。
  - **目的**：增强对模板文本、群发消息、刷屏行为的识别能力。

最终生成的每条文本向量由以下部分组成：(语义部分的维度可以随着PCA进行调整，这里以256为例)

| 特征类型      | 描述              | 维度         |
|-----------|-----------------|------------|
| 语义部分      | 平均字向量（PCA降维后）   | 256        |
| 符号特征      | 特殊符号占比、符号数量     | 2          |
| 分布特征      | 信息熵、自相似性        | 2          |
| <b>合计</b> | <b>最终特征向量维度</b> | <b>260</b> |

输出数组 `x.shape = (样本数, 特征维度)`，可直接用于神经网络等分类器输入。

```

1 def text_to_embedding(text, pca, char_embeddings, char2idx):
2     # 初始化PCA字向量的均值（语义部分）
3     vec_semantic = np.zeros(pca.n_components_)
4     # 1. 提取有效汉字并计算语义向量
5     valid_chars = [char for char in text if char in char2idx] # 仅处理字典中的汉字
6     if valid_chars:
7         indices = [char2idx[char] for char in valid_chars]
8         vec_semantic = np.mean(char_embeddings[indices], axis=0) # 字向量均值
9     # 2. 计算特殊符号占比特征（结构部分）

```

```

10     special_chars = re.findall(r'[\u4e00-\u9fa5a-zA-Z0-9]', text) #
    排除空格
11     special_ratio = len(special_chars) / max(1, len(text)) # 符号占比
12     # 3. 融合语义向量和符号特征
13     vec_combined = np.concatenate([
14         vec_semantic,                # PCA字向量均值（语义）
15         [special_ratio],             # 特殊符号占比（结构）
16         [len(special_chars)],        # 绝对符号数量
17         [self_similarity(text)],     # 自相似性
18         [text_entropy(text)],        # 信息熵
19     ])
20     return vec_combined

```

## 2.5 分类器

分类器以文本向量作为特征

为了适配不同计算资源的场景，我们实现了CPU版本和GPU版本的分类器，二者均为两层全连接层构建的神经网络，但是在细节上不一样，下面以GPU版本为例：

### 2.5.1 模型结构

使用 PyTorch 构建了一个简洁的 **多层感知机（MLP）分类器**，具备以下结构：

- 输入层：维度与文本特征向量一致（如 304）
- 隐藏层1：
  - 全连接层（默认维度 256）
  - Batch Normalization
  - ReLU 激活函数
  - Dropout (0.4) 用于正则化
- 隐藏层2：
  - 全连接层（默认维度 128）
  - Batch Normalization
  - ReLU 激活函数
  - Dropout (0.4)
- 输出层：
  - 全连接层输出类别数（默认2类：垃圾 / 非垃圾）



```
1 MLP(  
2     Linear(input_dim → 256)  
3     → BatchNorm → ReLU → Dropout  
4     → Linear(256 → 128)  
5     → BatchNorm → ReLU → Dropout  
6     → Linear(128 → 2)  
7 )
```

### 2.5.2 训练集样本平衡与划分

为解决实际场景中垃圾文本样本与正常文本样本数量不均的问题，采用 **随机过采样 (RandomOverSampler)** 策略，在训练集上对少数类样本进行复制，从而平衡正负样本比例，避免模型偏向多数类。

- 原始标签通过 `LabelEncoder` 转换为数字编码，便于模型识别；
- 使用 `train_test_split` 划分训练集与测试集，并保持原始类别比例 (`stratify=y`)；
- 针对训练集执行 `RandomOverSampler` 采样操作，指定 `sampling_strategy=0.7`，即：
  - 将少数类的样本数提升到多数类的 70%，避免过度复制；
- 输出采样前后每个类别的样本数量，方便核查是否平衡成功。

```
1 oversampler = RandomOverSampler(sampling_strategy=0.7, random_state=42)  
2 X_train, y_train = oversampler.fit_resample(X_train, y_train)
```

通过样本平衡与划分，可以进一步增强模型对少数类（如“垃圾文本”）的学习能力，同时提高召回率，降低因数据偏斜造成的漏检。

### 2.5.3 训练与推理配置

- 优化器和损失函数
  - `Adam` 优化器（学习率：1e-3）
  - 损失函数使用 `CrossEntropyLoss`，支持多类分类；
  - 可扩展为加权损失以应对类别不平衡（支持 `class_weights`）。
- 训练集标准化
  - 使用 `StandardScaler` 对特征向量进行零均值单位方差归一化；
  - 保证各维度在统一尺度下训练，提升梯度收敛效率。
- 训练过程设置
  - 设置随机种子保证结果可复现；
  - 每轮训练结束输出：Loss、训练集准确率、测试集准确率；
- 模型评估与保存
  - 每轮训练后在测试集评估性能；
  - 训练完成后使用 `classification_report` 和 `confusion_matrix` 输出完整评估指标；
  - 最终模型以 `.pth` 格式保存至磁盘，可用于后续推理或部署；

2.5.4 分类器配置表

| 项目                   | 设置值                   |
|----------------------|-----------------------|
| 隐藏层1维度               | 256                   |
| 隐藏层2维度               | 128                   |
| 激活函数                 | ReLU                  |
| Dropout比例            | 0.4                   |
| 优化器                  | Adam                  |
| 学习率                  | 1e-3                  |
| Loss函数               | CrossEntropyLoss      |
| Epochs               | 40                    |
| Batch Size           | 1024                  |
| 归一化方式                | StandardScaler        |
| RandomOverSampler 策略 | sampling_strategy=0.7 |

总的来说，分类器的架构优点如下：

| 特性    | 说明                               |
|-------|----------------------------------|
| 轻量可控  | 网络结构简洁，推理速度快，适合在线服务部署            |
| GPU优化 | 多线程加载 + 显存固定 + 异步张量传输，训练效率高      |
| 易于拓展  | 可快速集成 Transformer、BERT 等模块替换 MLP |

```
1 #####GPU训练
2 # 设置随机种子
3 torch.manual_seed(42)
4 np.random.seed(42)
5 load_existing_model=False
6 # 归一化
7 scaler = StandardScaler()
8 X_train = scaler.fit_transform(X_train)
9 X_test = scaler.transform(X_test)
10 # 转为 Tensor
11 X_train_tensor = torch.tensor(X_train, dtype=torch.float32)
12 y_train_tensor = torch.tensor(y_train, dtype=torch.long)
13 X_test_tensor = torch.tensor(X_test, dtype=torch.float32)
14 y_test_tensor = torch.tensor(y_test, dtype=torch.long)
15 # 数据加载器
16 train_loader = DataLoader(
17     TensorDataset(X_train_tensor, y_train_tensor),
```

```

18 batch_size=1024,
19 shuffle=True,
20 num_workers=16, # 加速数据加载
21 pin_memory=True, # 推荐用于 GPU 模型
22 prefetch_factor=8,
23 persistent_workers=True #避免反复启动线程，提高训练速度
24 )
25 # 定义 MLP 模型
26 class MLP(nn.Module):
27     def __init__(self, input_dim, hidden1=256, hidden2=128,
28                 output_dim=2):
29         super().__init__()
30         self.net = nn.Sequential(
31             nn.Linear(input_dim, hidden1),
32             nn.BatchNorm1d(hidden1),
33             nn.ReLU(),
34             nn.Dropout(0.4), # 添加 dropout
35             nn.Linear(hidden1, hidden2),
36             nn.BatchNorm1d(hidden2),
37             nn.ReLU(),
38             nn.Dropout(0.4),
39             nn.Linear(hidden2, output_dim)
40         )
41     def forward(self, x):
42         return self.net(x)
43 model = MLP(input_dim=X.shape[1],
44             output_dim=len(le.classes_)).to(device)
45 # if load_existing_model:
46 #     load_model(model)
47 #model = torch.compile(model) #编译，以进一步提高速度
48 # 损失函数与优化器
49 criterion = nn.CrossEntropyLoss()
50 optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)

```

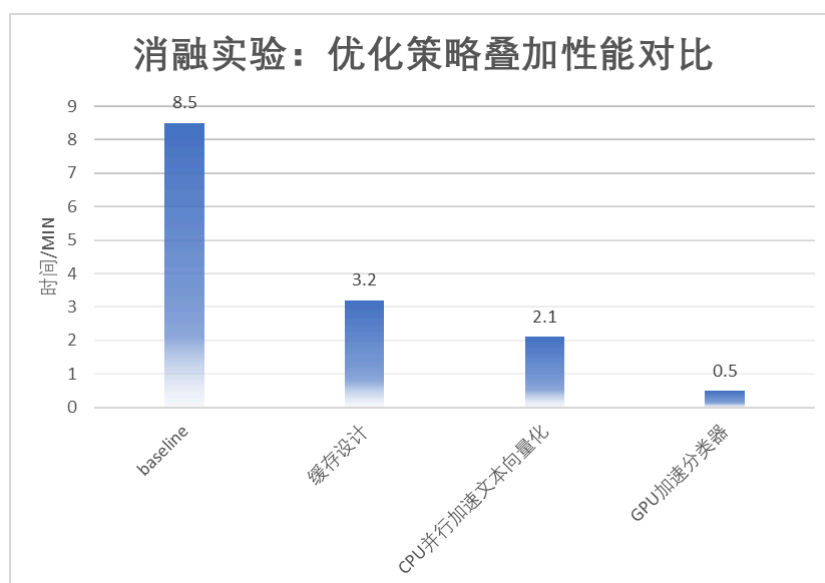
## 三，系统优化设计

为提高性能，进一步优化算法中某些流程的性能，追求并行扩展性。同时，为进一步适应环境，配置了资源受限的版本。为提高用户体验，设计了前端系统。

### 3.1 性能优化

- 字符向量与编码
  - 为了避免重复计算中文字符的编码以及相似性矩阵，我们设置了缓存机制，即将字符编码和相似性矩阵以一个文件的方式缓存下来，下次训练/推理时直接调用该文件，避免重复计算。

- PCA维度的调整为256维，保留97%的信息，同时尽可能大的压缩维度。（不压缩为8000维）
- 文本向量与特征工程
  - 使用 `joblib.Parallel(n_jobs=32)` 实现批量向量生成的并行加速处理，提升大型数据集处理效率。
- 分类器
  - 数据加载优化（`DataLoader`）：
    - 启用多线程（`num_workers=16`）与 `pin_memory=True`；
    - 使用 `prefetch_factor` 和 `persistent_workers=True` 提高 GPU 数据流效率。
  - 训练过程优化
    - 支持异步张量传输到 GPU： `non_blocking=True`；
  - 推理过程优化
    - 可选开启模型编译加速（如使用 `torch.compile()`）。



## 3.2 轻量化版本设计

为了适应不同的计算场景，我们还设计了轻量化的训练和推理版本，其改动如下：

- 轻量化相似度矩阵（端侧只使用5k常用字，云侧高性能版本使用8k字）
- 分类器模型弱化
- PCA维度降低-128维

## 3.3 前端配套设计

为了更好的展示以及为推理提供接口，我们设计了一套前端UI。视图将其集成成一个易用的APP，方便部署到云端和本地。其大致流程如下

用户按下“检测”按钮后，网页通过POST请求向后端发送“/predict”请求，并将待检测文本编码后放入请求体发送。后端收到“/predict”请求后对待检测文本进行解码，使用模型训练时保存的字嵌入矩阵进行文本向量化，并通过保存的模型进行预测，并计算每个字符对于判断的贡献值。预测完成后，后端将预测结果，结果置信度，每个字符贡献值数组，平均贡献值封装为json格式响应

并返回网页。网页解析后输出预测结果，并通过读取贡献值数组对被检测文本中贡献值大于平均贡献值的字符进行标注。

POST请求结构如下

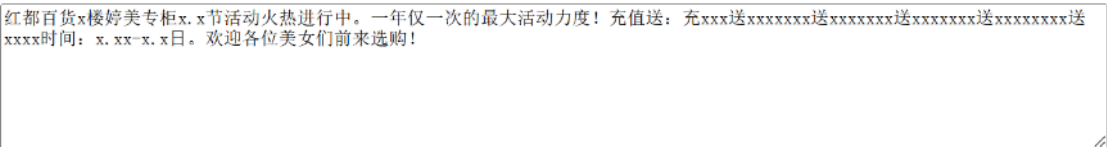
```
1 method: 'POST',
2 headers: { 'Content-Type': 'application/x-www-form-urlencoded' },
3 body: `text=${encodeURIComponent(text)}`
```

响应结构如下

```
1 'label': LABEL_MAPPING.get(pred, '未知'),
2 'probability': float(proba[pred]),
3 'contributions': contributions,
4 'avg': avg_score
```

效果图如下：

垃圾信息检测器



检测

分类：垃圾信息（置信度：100.0%）

红都百货x楼婷美专柜x.x节活动火热进行中。一年仅一次的最大活动力度！充值送：充xxx送xxxxxxx送xxxxxxx送xxxxxxx送xxxxx时间：x.xx-x.x日。欢迎各位美女们前来选购！

四，实验结果与分析

4.1 测试方案与指标说明

评价算法和模型的指标如下：

| 指标             | 含义                                                 |
|----------------|----------------------------------------------------|
| Precision（精确率） | 正确预测为某类的样本数占有所有预测为该类的样本数的比例。即：模型说“是”的时候，有多大概率是对的。  |
| Recall（召回率）    | 正确预测为某类的样本数占有所有真实为该类的样本数的比例。即：所有应该被找出来的，有多少真的找出来了。 |
| F1-score       | 精确率和召回率的调和平均值，用于权衡两者。高F1分说明模型在保持准确的同时也没有漏掉太多。      |

在三个数据集上训练并测试

数据集1，助教给的数据集，一共16k数据，其中垃圾文本tag为1，非垃圾文本tag为0。正常文本：垃圾文本=1：2。

数据集2，网上自己找的数据集，一共800k数据，其中垃圾文本tag为1，非垃圾文本tag为0。正常文本：垃圾文本=10:1。

混合数据集：混合数据集1和数据集2，一共32k数据，其中一半来自数据集1，一般来自数据集2。正常文本：垃圾文本=1：1。

## 4.2 效果展示

数据集1-16k

|       |              |           |        |          |         |
|-------|--------------|-----------|--------|----------|---------|
| 分类报告： |              |           |        |          |         |
|       |              | precision | recall | f1-score | support |
|       | 0            | 0.9900    | 0.9910 | 0.9905   | 1000    |
|       | 1            | 0.9959    | 0.9955 | 0.9957   | 2200    |
|       | accuracy     |           |        | 0.9941   | 3200    |
|       | macro avg    | 0.9930    | 0.9932 | 0.9931   | 3200    |
|       | weighted avg | 0.9941    | 0.9941 | 0.9941   | 3200    |
| 混淆矩阵： |              |           |        |          |         |
|       | [[ 991 9]    |           |        |          |         |
|       | [ 10 2190]]  |           |        |          |         |

|    |             |              |        |          |         |      |
|----|-------------|--------------|--------|----------|---------|------|
| 1  | 分类报告：       |              |        |          |         |      |
| 2  |             | precision    | recall | f1-score | support |      |
| 3  |             |              |        |          |         |      |
| 4  |             | 0            | 0.9900 | 0.9910   | 0.9905  | 1000 |
| 5  |             | 1            | 0.9959 | 0.9955   | 0.9957  | 2200 |
| 6  |             |              |        |          |         |      |
| 7  |             | accuracy     |        |          | 0.9941  | 3200 |
| 8  |             | macro avg    | 0.9930 | 0.9932   | 0.9931  | 3200 |
| 9  |             | weighted avg | 0.9941 | 0.9941   | 0.9941  | 3200 |
| 10 |             |              |        |          |         |      |
| 11 |             |              |        |          |         |      |
| 12 | 混淆矩阵：       |              |        |          |         |      |
| 13 | [[ 991 9]   |              |        |          |         |      |
| 14 | [ 10 2190]] |              |        |          |         |      |

数据集2-800k

|       |               |           |        |          |         |
|-------|---------------|-----------|--------|----------|---------|
| 分类报告： |               |           |        |          |         |
|       |               | precision | recall | f1-score | support |
|       | 0             | 0.9971    | 0.9969 | 0.9970   | 144000  |
|       | 1             | 0.9725    | 0.9736 | 0.9730   | 16000   |
|       | accuracy      |           |        | 0.9946   | 160000  |
|       | macro avg     | 0.9848    | 0.9852 | 0.9850   | 160000  |
|       | weighted avg  | 0.9946    | 0.9946 | 0.9946   | 160000  |
| 混淆矩阵： |               |           |        |          |         |
|       | [[143559 441] |           |        |          |         |
|       | [ 423 15577]] |           |        |          |         |

|   |       |           |        |          |         |
|---|-------|-----------|--------|----------|---------|
| 1 | 分类报告： |           |        |          |         |
| 2 |       | precision | recall | f1-score | support |
| 3 |       |           |        |          |         |

|    |               |        |        |        |        |
|----|---------------|--------|--------|--------|--------|
| 4  | 0             | 0.9971 | 0.9969 | 0.9970 | 144000 |
| 5  | 1             | 0.9725 | 0.9736 | 0.9730 | 16000  |
| 6  |               |        |        |        |        |
| 7  | accuracy      |        |        | 0.9946 | 160000 |
| 8  | macro avg     | 0.9848 | 0.9852 | 0.9850 | 160000 |
| 9  | weighted avg  | 0.9946 | 0.9946 | 0.9946 | 160000 |
| 10 |               |        |        |        |        |
| 11 |               |        |        |        |        |
| 12 | 混淆矩阵：         |        |        |        |        |
| 13 | [[143559 441] |        |        |        |        |
| 14 | [ 423 15577]] |        |        |        |        |

数据集3（混合数据集）-32k

|             |              |           |        |          |         |
|-------------|--------------|-----------|--------|----------|---------|
| 分类报告：       |              |           |        |          |         |
|             |              | precision | recall | f1-score | support |
|             | 0            | 0.9783    | 0.9874 | 0.9828   | 3884    |
|             | 1            | 0.9802    | 0.9662 | 0.9732   | 2516    |
|             |              |           |        |          |         |
|             | accuracy     |           |        | 0.9791   | 6400    |
|             | macro avg    | 0.9793    | 0.9768 | 0.9780   | 6400    |
|             | weighted avg | 0.9791    | 0.9791 | 0.9790   | 6400    |
| 混淆矩阵：       |              |           |        |          |         |
| [[3835 49]  |              |           |        |          |         |
| [ 85 2431]] |              |           |        |          |         |

|    |              |           |        |          |         |
|----|--------------|-----------|--------|----------|---------|
| 1  | 分类报告：        |           |        |          |         |
| 2  |              | precision | recall | f1-score | support |
| 3  |              |           |        |          |         |
| 4  | 0            | 0.9783    | 0.9874 | 0.9828   | 3884    |
| 5  | 1            | 0.9802    | 0.9662 | 0.9732   | 2516    |
| 6  |              |           |        |          |         |
| 7  | accuracy     |           |        | 0.9791   | 6400    |
| 8  | macro avg    | 0.9793    | 0.9768 | 0.9780   | 6400    |
| 9  | weighted avg | 0.9791    | 0.9791 | 0.9790   | 6400    |
| 10 |              |           |        |          |         |
| 11 |              |           |        |          |         |
| 12 | 混淆矩阵：        |           |        |          |         |
| 13 | [[3835 49]   |           |        |          |         |
| 14 | [ 85 2431]]  |           |        |          |         |

## 4.3 结果分析

- 数据集1
  - 表现非常良好，正类负类都达到了一个非常高的指标。
  - 仅9个正常文本被误判（假阳性），10个垃圾文本漏判（假阴性）
  - 适用于对误报容忍度低的场景（如安全审核）
- 数据集2
  - 非垃圾文本的正确率和召回率都非常高，但是垃圾文本的f1-score略低（97.17%），因为数据集2大都是正常文本（比例为10：1），样本极端不平衡导致少数类表现下降。

- 441个正常文本误判（0.3%），但15577个垃圾文本正确识别
- 适合"宁可错杀不可放过"的场景（如垃圾邮件过滤）
- 数据集3
  - 在三类数据集中表现最差，因为不同数据集有自己的特征。但是该数据集下泛化效果较好，泛化性最佳但精度略降。
  - 49/3884正常文本误判（1.26%），85/2516垃圾文本漏判（3.38%）
  - 平衡性最佳，适合通用场景
- 数据分布对模型影响显著，数据集1的均衡分布带来最优表现
- 混合数据虽牺牲2-3%的精度，但更适合实际场景（兼顾不同数据特性）

## 4.4 消融实验与分析

**完整模型对比** 完整模型 vs 移除目标组件的模型 验证单一组件的独立贡献。为求方便以macro avg中的f1-score作为统一评估指标，这样可以加强少数类的影响。

### 4.4.1 无结构特征 vs 有结构特征（特征工程）

这里比较额外特征的效果，主要有特殊符号、自相似性、信息熵等结构特征。

|   |              |           |        |          |         |
|---|--------------|-----------|--------|----------|---------|
| 1 | 16k数据集-无结构特征 |           |        |          |         |
| 2 |              | precision | recall | f1-score | support |
| 3 |              |           |        |          |         |
| 4 | 0            | 0.9820    | 0.9800 | 0.9810   | 1000    |
| 5 | 1            | 0.9909    | 0.9918 | 0.9914   | 2200    |
| 6 |              |           |        |          |         |
| 7 | accuracy     |           |        | 0.9881   | 3200    |
| 8 | macro avg    | 0.9864    | 0.9859 | 0.9862   | 3200    |
| 9 | weighted avg | 0.9881    | 0.9881 | 0.9881   | 3200    |

|    |               |           |        |          |         |
|----|---------------|-----------|--------|----------|---------|
| 1  | 800k数据集-无结构特征 |           |        |          |         |
| 2  | 分类报告：         |           |        |          |         |
| 3  |               | precision | recall | f1-score | support |
| 4  |               |           |        |          |         |
| 5  | 0             | 0.9950    | 0.9926 | 0.9938   | 144000  |
| 6  | 1             | 0.9350    | 0.9555 | 0.9452   | 16000   |
| 7  |               |           |        |          |         |
| 8  | accuracy      |           |        | 0.9889   | 160000  |
| 9  | macro avg     | 0.9650    | 0.9741 | 0.9695   | 160000  |
| 10 | weighted avg  | 0.9890    | 0.9889 | 0.9890   | 160000  |

|         | 无结构特征  | 有结构特征  |
|---------|--------|--------|
| 16k数据集  | 0.9881 | 0.9941 |
| 800k数据集 | 0.9889 | 0.9946 |



- 新增额外结构特征，对模型的优化效果显著
- 因为垃圾文本数据集中，垃圾文本一般含有大量的特殊字符，广告之类的文本还会具有一定的重复性，因此这些结构特征可以较好的用于分类。

#### 4.4.2 样本平衡

此处比较样本平衡的效果。

|    |              |           |        |          |         |
|----|--------------|-----------|--------|----------|---------|
| 1  | 16k-不平衡      |           |        |          |         |
| 2  | 分类报告：        |           |        |          |         |
| 3  |              | precision | recall | f1-score | support |
| 4  |              |           |        |          |         |
| 5  | 0            | 0.9890    | 0.9880 | 0.9885   | 1000    |
| 6  | 1            | 0.9945    | 0.9950 | 0.9948   | 2200    |
| 7  |              |           |        |          |         |
| 8  | accuracy     |           |        | 0.9928   | 3200    |
| 9  | macro avg    | 0.9918    | 0.9915 | 0.9916   | 3200    |
| 10 | weighted avg | 0.9928    | 0.9928 | 0.9928   | 3200    |

|    |              |           |        |          |         |
|----|--------------|-----------|--------|----------|---------|
| 1  | 800k-样本不平衡   |           |        |          |         |
| 2  | 分类报告：        |           |        |          |         |
| 3  |              | precision | recall | f1-score | support |
| 4  |              |           |        |          |         |
| 5  | 0            | 0.9971    | 0.9967 | 0.9969   | 144000  |
| 6  | 1            | 0.9700    | 0.9742 | 0.9721   | 16000   |
| 7  |              |           |        |          |         |
| 8  | accuracy     |           |        | 0.9944   | 160000  |
| 9  | macro avg    | 0.9836    | 0.9855 | 0.9845   | 160000  |
| 10 | weighted avg | 0.9944    | 0.9944 | 0.9944   | 160000  |

|         | 样本不平衡  | 样本平衡   |
|---------|--------|--------|
| 16k数据集  | 0.9928 | 0.9941 |
| 800k数据集 | 0.9944 | 0.9946 |

- 样本平衡后，少数类的分类效果得到较大的提升。
- 小数据集对样本平衡似乎更加敏感。

#### 4.4.3 低维PCA vs 高维PCA

此处进行不同PCA维度对模型效果的影响。

|    |              |           |        |          |         |  |
|----|--------------|-----------|--------|----------|---------|--|
| 1  | 800k-128     |           |        |          |         |  |
| 2  | 分类报告：        |           |        |          |         |  |
| 3  |              | precision | recall | f1-score | support |  |
| 4  |              |           |        |          |         |  |
| 5  | 0            | 0.9973    | 0.9940 | 0.9957   | 144000  |  |
| 6  | 1            | 0.9477    | 0.9761 | 0.9617   | 16000   |  |
| 7  |              |           |        |          |         |  |
| 8  | accuracy     |           |        | 0.9922   | 160000  |  |
| 9  | macro avg    | 0.9725    | 0.9851 | 0.9787   | 160000  |  |
| 10 | weighted avg | 0.9924    | 0.9922 | 0.9923   | 160000  |  |

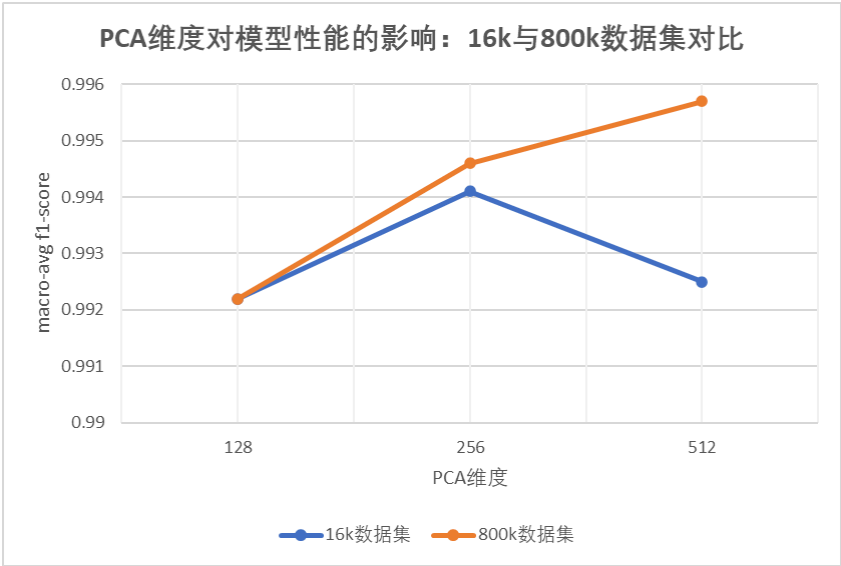
|    |              |           |        |          |         |  |
|----|--------------|-----------|--------|----------|---------|--|
| 1  | 800k-512     |           |        |          |         |  |
| 2  | 分类报告：        |           |        |          |         |  |
| 3  |              | precision | recall | f1-score | support |  |
| 4  |              |           |        |          |         |  |
| 5  | 0            | 0.9975    | 0.9978 | 0.9976   | 144000  |  |
| 6  | 1            | 0.9801    | 0.9772 | 0.9787   | 16000   |  |
| 7  |              |           |        |          |         |  |
| 8  | accuracy     |           |        | 0.9957   | 160000  |  |
| 9  | macro avg    | 0.9888    | 0.9875 | 0.9881   | 160000  |  |
| 10 | weighted avg | 0.9957    | 0.9957 | 0.9957   | 160000  |  |

|    |              |           |        |          |         |  |
|----|--------------|-----------|--------|----------|---------|--|
| 1  | 16k-128      |           |        |          |         |  |
| 2  | 分类报告：        |           |        |          |         |  |
| 3  |              | precision | recall | f1-score | support |  |
| 4  |              |           |        |          |         |  |
| 5  | 0            | 0.9939    | 0.9810 | 0.9874   | 1000    |  |
| 6  | 1            | 0.9914    | 0.9973 | 0.9943   | 2200    |  |
| 7  |              |           |        |          |         |  |
| 8  | accuracy     |           |        | 0.9922   | 3200    |  |
| 9  | macro avg    | 0.9927    | 0.9891 | 0.9909   | 3200    |  |
| 10 | weighted avg | 0.9922    | 0.9922 | 0.9922   | 3200    |  |

|    |              |           |        |          |         |  |
|----|--------------|-----------|--------|----------|---------|--|
| 1  | 16k-512      |           |        |          |         |  |
| 2  | 分类报告：        |           |        |          |         |  |
| 3  |              | precision | recall | f1-score | support |  |
| 4  |              |           |        |          |         |  |
| 5  | 0            | 0.9900    | 0.9860 | 0.9880   | 1000    |  |
| 6  | 1            | 0.9936    | 0.9955 | 0.9946   | 2200    |  |
| 7  |              |           |        |          |         |  |
| 8  | accuracy     |           |        | 0.9925   | 3200    |  |
| 9  | macro avg    | 0.9918    | 0.9907 | 0.9913   | 3200    |  |
| 10 | weighted avg | 0.9925    | 0.9925 | 0.9925   | 3200    |  |

| PCA维度 | 16k数据集 | 800k数据集 |
|-------|--------|---------|
| 128   | 0.9922 | 0.9922  |
| 256   | 0.9941 | 0.9946  |

| PCA维度 | 16k数据集 | 800k数据集 |
|-------|--------|---------|
| 512   | 0.9925 | 0.9957  |



- 一般而言，PCA保留的维度越多，信息越多，分类的正确性越好。
- 但是维度过多容易导致性能下降。256是一个平衡点。

#### 4.4.4 普通逻辑回归分类器和神经网络分类器

此处针对分类器的复杂程度进行对比，以macro-avg f1-score为指标，主要比对逻辑回归和两成全连接神经网络。

|    |              |           |        |          |         |
|----|--------------|-----------|--------|----------|---------|
| 1  | 16k-逻辑回归     |           |        |          |         |
| 2  | 分类报告：        |           |        |          |         |
| 3  |              | precision | recall | f1-score | support |
| 4  |              |           |        |          |         |
| 5  | 0            | 0.9722    | 0.9780 | 0.9751   | 1000    |
| 6  | 1            | 0.9900    | 0.9873 | 0.9886   | 2200    |
| 7  |              |           |        |          |         |
| 8  | accuracy     |           |        | 0.9844   | 3200    |
| 9  | macro avg    | 0.9811    | 0.9826 | 0.9818   | 3200    |
| 10 | weighted avg | 0.9844    | 0.9844 | 0.9844   | 3200    |

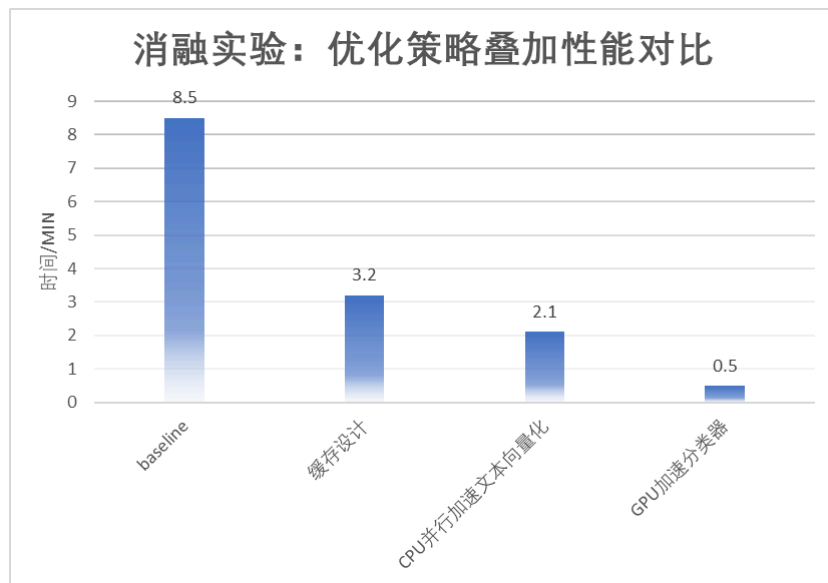
|    |              |           |        |          |         |
|----|--------------|-----------|--------|----------|---------|
| 1  | 800k-逻辑回归    |           |        |          |         |
| 2  | 分类报告：        |           |        |          |         |
| 3  |              | precision | recall | f1-score | support |
| 4  |              |           |        |          |         |
| 5  | 0            | 0.9957    | 0.9803 | 0.9880   | 14385   |
| 6  | 1            | 0.8459    | 0.9622 | 0.9003   | 1615    |
| 7  |              |           |        |          |         |
| 8  | accuracy     |           |        | 0.9785   | 16000   |
| 9  | macro avg    | 0.9208    | 0.9713 | 0.9441   | 16000   |
| 10 | weighted avg | 0.9806    | 0.9785 | 0.9791   | 16000   |

|               | 逻辑回归      | 两层全连接     |
|---------------|-----------|-----------|
| 16k的f1-score  | 0.9844    | 0.9941    |
| 800k的f1-score | 0.9785    | 0.9946    |
| 16k的训练用时      | 10s (CPU) | 20s (GPU) |

- 更换较为复杂的分类器模型对于整个模型的优化效果显著。
- 两层全连接分类器显著优于逻辑回归
- 在一定的优化技巧下，两层全连接分类器的训练用时较逻辑回归仅多一倍。

#### 4.4.5 优化性能用时（端侧版本和高性能CPU+GPU版本）

- baseline（以16k数据集为例）
- 缓存设计
- CPU并行加速文本向量化
- GPU加速分类器



#### 4.4.6 端侧训练推理模型设计

- 轻量化缓存相似度矩阵（端侧只是用4k常用字）
- 分类器模型弱化
- PCA维度降低-128维

这里采用16k数据集作为测试。

|    |              |           |        |          |         |
|----|--------------|-----------|--------|----------|---------|
| 1  | 16k数据集效果     |           |        |          |         |
| 2  | 分类报告：        |           |        |          |         |
| 3  |              | precision | recall | f1-score | support |
| 4  |              |           |        |          |         |
| 5  | 0            | 0.9879    | 0.9810 | 0.9844   | 1000    |
| 6  | 1            | 0.9914    | 0.9946 | 0.9930   | 2202    |
| 7  |              |           |        |          |         |
| 8  | accuracy     |           |        | 0.9903   | 3202    |
| 9  | macro avg    | 0.9897    | 0.9878 | 0.9887   | 3202    |
| 10 | weighted avg | 0.9903    | 0.9903 | 0.9903   | 3202    |

|                    | 端侧（性能受限）    | 云侧（高性能）     |
|--------------------|-------------|-------------|
| 内存                 | 300MB       | 600MB       |
| 完整训练时长             | 17s (使用CPU) | 28s (使用GPU) |
| macro avg f1-score | 0.9903      | 0.9941      |

## 五，创新点与总结

### 1，创新的特征表示方法

- 融合汉字结构特征（字形、四角编码）与语义特征
- 引入文本自相似性和信息熵等统计特征
- 基于PCA降维的紧凑向量表示

### 2，高性能计算架构

- 实现了缓存机制加速特征计算
- 支持多核并行文本向量化处理
- 提供CPU/GPU双版本分类器，并对训练过程进行了一定的优化

### 3，优异的分类性能

- 在16k数据集上达到99.41%算术平均f1-score
- 在800k大规模数据集上保持99.46%算数平均f1-score
- 对混合数据集展现良好泛化能力

### 4，优秀的鲁棒性

- 针对数据集未编码字符OOV进行了针对化处理。针对数据集正负样本失衡，采用了 `RandomOverSampler` 的设计。

### 5，完善的工程实现

- 用户友好型。增强了可视化能力，提供可视化Web界面进行测试，可以更加直观的评估模型性能。

- **支持端侧轻量化部署。**针对资源受限场景，在保持高分类正确性的前提下减小计算开销与内存开销。
- 具备完整的训练/推理流水线

本系统可广泛应用于：

- 电子邮件垃圾过滤
- 社交媒体内容审核
- 广告和欺诈信息识别
- 短文本分类相关场景

本项目实现了一个高效、鲁棒的中文垃圾文本分类系统，通过创新的特征工程和优化的计算框架，在多个数据集上实现了99%以上的准确率和召回率。其优异的性能和灵活的部署方式，使其能够适应从移动端到云计算平台的各种应用环境。未来可通过引入更先进的深度学习模型和持续学习机制进一步提升系统能力。

## 可能的进一步优化点

- 使用动态可学习的attention对字/词向量进行聚合，生成上下文关系更为密切的文本向量。  
(实验发现静态attention效果甚至不如普通的平均池化生成文本向量)
- 本系统没有进行分词，实现分词后再映射为词向量，应该会比字向量映射更为有效。